AIML ENGINEERING LAB — P053 REFERENCE GUIDE

Advanced PyTorch Principal Engineer's Complete Guide

Everything you need to know for AMD, NVIDIA, Micron, Qualcomm interviews.
From tensors to production deployment — with real P053 code examples.

Last Updated: April 2026 · PyTorch 2.x · CUDA 12.1 · Python 3.12

Table of Contents

1. What is PyTorch? History, Philosophy, and Why It Won

1.1 PyTorch vs TensorFlow — The War That Shaped ML

1.2 Who Uses PyTorch? (Companies, Research, Industry)

1.3 PyTorch Ecosystem Map

2. Tensors — The Foundation of Everything

2.1 Creating Tensors (14 Ways)

2.2 Tensor Operations, Broadcasting, Indexing

2.3 Device Management (CPU → GPU → MPS)

2.4 Memory Layout (contiguous, stride, view vs clone)

3. Autograd — Automatic Differentiation Engine

3.1 Computation Graph (Dynamic vs Static)

3.2 Gradient Accumulation, Retention, and Zeroing

3.3 Custom Autograd Functions

4. nn.Module — Building Neural Networks

4.1 Layer Types (Linear, Conv, Attention, Embedding)

4.2 Activation Functions (ReLU, GELU, SiLU, Mish)

4.3 Custom Modules and Forward Hooks

4.4 P053 HybridTransformerCNN Architecture Deep Dive

5. Training Loop — The Heart of Deep Learning

5.1 Loss Functions (BCE, CrossEntropy, FocalLoss, Custom)

5.2 Optimizers (SGD, Adam, AdamW, LAMB, LARS)

5.3 Learning Rate Schedulers (Step, Cosine, OneCycle, Warmup)

5.4 P053 Training Loop — Line by Line

6. Data Pipeline — DataLoader, Dataset, Samplers

6.1 Custom Dataset Classes

6.2 DataLoader Internals (num_workers, prefetch, pin_memory)

6.3 Handling Class Imbalance (WeightedRandomSampler)

7. GPU Training — CUDA, AMP, Multi-GPU

7.1 CUDA Fundamentals for ML Engineers

7.2 Mixed Precision (AMP) — float16 vs bfloat16

7.3 GradScaler Deep Dive + P053 Death Spiral Bug

7.4 Multi-GPU (DataParallel, DDP, FSDP)

7.5 Apple MPS Backend

8. Model Saving, Loading, and Deployment

8.1 state_dict vs Entire Model Save

8.2 TorchScript and torch.jit

8.3 ONNX Export

8.4 Quantization (INT8, INT4, QAT)

9. PyTorch 2.x — torch.compile and Beyond

9.1 TorchDynamo and TorchInductor

9.2 torch.compile() — When and How to Use

9.3 Triton Integration (NVIDIA)

10. Advanced Patterns for Production

10.1 Gradient Checkpointing (Memory vs Compute)

10.2 Custom CUDA Kernels

10.3 Profiling with torch.profiler

10.4 Debugging NaN/Inf Gradients

11. PyTorch at AMD, NVIDIA, Qualcomm, Micron

12. Complete API Reference — Most Used Functions

13. Interview Questions with Answers

1. What is PyTorch? History, Philosophy, and Why It Won

PyTorch is an open-source machine learning framework developed by **Meta AI (Facebook AI Research / FAIR)**. It was released in January 2017, built on top of the Torch library (Lua-based, developed at NYU). PyTorch provides two core capabilities:

1. **Tensor computation** with GPU acceleration (like NumPy but on CUDA)
2. **Automatic differentiation** (autograd) for building and training neural networks

> **Key philosophy:** "Define-by-run" — the computation graph is built dynamically as you execute code. This means PyTorch code looks like regular Python. You can use `if`, `for`, `while`, `print()` freely. TensorFlow 1.x required you to define the entire graph before running it. This one decision won PyTorch the research community.

1.1 PyTorch vs TensorFlow — The War That Shaped ML

Feature	PyTorch	TensorFlow
Creator	Meta AI (FAIR)	Google Brain
Graph type	Dynamic (eager execution)	Static (TF1) / Eager (TF2)
Debugging	Standard Python debugger (pdb, breakpoint())	tf.debugging tools, harder to step through
Research adoption	~80% of ML papers at NeurIPS/ICML 2024	~15% and declining
Production adoption	Growing fast (TorchServe, TorchScript, ONNX)	Strong (TF Serving, TF Lite, TF.js)
Mobile/Edge	ExecuTorch (new), PyTorch Mobile	TF Lite (mature, wide adoption)
Compilation	torch.compile (TorchDynamo + Inductor)	XLA compiler
GPU support	CUDA, ROCm (AMD), MPS (Apple)	CUDA, TPU (Google)
Distributed training	DDP, FSDP (native)	tf.distribute (MirroredStrategy)
Ecosystem	Hugging Face, Lightning, timm, detectron2	Keras, TF Hub, TF Extended (TFX)
Our P053 choice	✅ **Selected** — dynamic graph for custom architecture + AMP control	❌ Rejected — less control over mixed precision dtype

> **Interview answer:** "We chose PyTorch for P053 because we needed fine-grained control over mixed precision (bfloat16 vs float16 per GPU), custom loss functions (FocalLoss), and a hybrid architecture that mixes Transformer + CNN. PyTorch's dynamic graph makes this feel like writing regular Python. TensorFlow could do it, but the debugging experience for custom autograd functions is significantly worse."

1.2 Who Uses PyTorch?

Company	Use Case	Why PyTorch
Meta	Recommendation, LLaMA, SAM	Created it. All internal ML on PyTorch.
NVIDIA	Megatron-LM, NeMo, CUDA integration	Deep CUDA integration, Tensor Core optimization
AMD	ROCm backend, Instinct MI300X	ROCm support makes PyTorch run on AMD GPUs
Tesla	Autopilot vision, Dojo training	Custom hardware + dynamic research pace
OpenAI	GPT-4, DALL-E, Whisper	Research flexibility for novel architectures
Microsoft	Bing, Copilot, DeepSpeed	DeepSpeed (distributed training) built on PyTorch
Hugging Face	Transformers library, 200K+ models	Community standard, model hub integration
Qualcomm	AI Engine, on-device inference	ONNX export from PyTorch → Qualcomm AI runtime
Micron / Samsung	Semiconductor yield, defect detection	Custom architectures for tabular + image data

1.3 PyTorch Ecosystem Map

Library	Purpose	Key Feature
`torch`	Core tensor + autograd	Foundation of everything
`torch.nn`	Neural network layers	Linear, Conv, Transformer, etc.
`torch.optim`	Optimizers	Adam, SGD, AdamW, LR schedulers
`torch.utils.data`	Data loading	Dataset, DataLoader, Sampler
`torch.amp`	Mixed precision	autocast, GradScaler
`torch.distributed`	Multi-GPU/multi-node	DDP, FSDP, RPC
`torch.profiler`	Performance profiling	CPU/GPU trace, memory, TensorBoard
`torch.compile`	Graph compilation (2.x)	TorchDynamo + Inductor for 2x speedup
`torchvision`	Computer vision	Models, transforms, datasets
`torchaudio`	Audio processing	Spectrograms, codecs
`torchtext`	NLP utilities	Tokenizers, vocab (deprecated → HF)
PyTorch Lightning	Training framework	Removes boilerplate, auto-GPU
Hugging Face Transformers	Pre-trained models	200K+ models, fine-tuning
DeepSpeed (Microsoft)	Distributed training	ZeRO optimizer, pipeline parallel
ONNX Runtime	Inference optimization	Cross-platform deployment

2. Tensors — The Foundation of Everything

A **tensor** is a multi-dimensional array — the data structure that flows through every neural network. Think of it as a NumPy ndarray that can live on a GPU.

Dimensions	Name	Example
0	Scalar	`torch.tensor(3.14)`
1	Vector	`torch.tensor([1, 2, 3])` — a feature vector
2	Matrix	`torch.randn(128, 36)` — a batch of 128 samples × 36 features
3	3D Tensor	`torch.randn(32, 128, 36)` — batch × sequence × features
4	4D Tensor	`torch.randn(32, 3, 224, 224)` — batch × channels × H × W (images)

2.1 Creating Tensors (14 Ways)

```
import torch

# From data
x = torch.tensor([1.0, 2.0, 3.0])              # From list
x = torch.tensor([[1, 2], [3, 4]])               # 2D from nested list
x = torch.from_numpy(np_array)                    # From NumPy (shares m

# Factory functions
x = torch.zeros(3, 4)                             # All zeros
x = torch.ones(3, 4)                              # All ones
x = torch.full((3, 4), fill_value=7.0)           # All sevens
x = torch.eye(4)                                  # Identity matrix
x = torch.arange(0, 10, 2)                        # [0, 2, 4, 6, 8]
x = torch.linspace(0, 1, 100)                     # 100 points from 0 to

# Random
x = torch.randn(128, 36)                          # Normal distribution
x = torch.rand(128, 36)                           # Uniform [0, 1)
x = torch.randint(0, 10, (128,))                  # Random integers

# Like another tensor
x = torch.zeros_like(other_tensor)                # Same shape/dtype/dev
x = torch.randn_like(other_tensor)                # Random with same sha

# On specific device
x = torch.randn(128, 36, device='cuda')           # Directly on GPU
x = torch.randn(128, 36, device='mps')            # Apple Silicon
x = torch.randn(128, 36, dtype=torch.bfloat16)    # Specific dtype
```

> **Critical gotcha — `torch.from_numpy()` shares memory!** If you modify the NumPy array, the tensor changes too (and vice versa). Use `torch.tensor(np_array)` to copy, or `.clone()` after.

2.2 Tensor Operations, Broadcasting, Indexing

```
# Element-wise operations
a + b                    # Addition (element-wise)
a * b                    # Multiplication (element-wise, NOT matrix mult
a @ b                    # Matrix multiplication (same as torch.matmul)
torch.matmul(a, b)       # Matrix multiplication (explicit)

# Reduction
x.sum()                  # Sum all elements → scalar
x.mean(dim=0)            # Mean along dimension 0 (batch mean)
x.max(dim=1)             # Max along dimension 1 (returns values, indice
x.argmax(dim=1)          # Index of max along dimension 1

# Reshaping
x.view(32, -1)           # Reshape (must be contiguous)
x.reshape(32, -1)        # Reshape (always works, may copy)
x.permute(0, 2, 1)       # Swap dimensions (like numpy transpose)
x.unsqueeze(0)           # Add dimension at position 0
x.squeeze()              # Remove all dimensions of size 1

# Broadcasting (NumPy rules apply)
a = torch.randn(128, 36)   # (128, 36)
b = torch.randn(36)        # (36,) → broadcasts to (128, 36)
c = a + b                   # Works! b is broadcast along dim 0

# Indexing (Python-style)
x[0]                     # First row
x[:, 0]                  # First column
x[x > 0]                # Boolean indexing (elements > 0)
x[:, [0, 5, 10]]        # Fancy indexing (columns 0, 5, 10)
```

2.3 Device Management (CPU → GPU → MPS)

```
# Check availability
torch.cuda.is_available()       # True if NVIDIA GPU + CUDA
torch.backends.mps.is_available() # True if Apple Silicon

# Automatic device selection (P053 pattern)
def get_device():
    if torch.cuda.is_available():
        return torch.device('cuda')
    elif torch.backends.mps.is_available():
        return torch.device('mps')
    return torch.device('cpu')

device = get_device()

# Move tensors
x = x.to(device)                # Move to device
x = x.cuda()                    # Move to first CUDA GPU
x = x.cpu()                     # Move back to CPU

# Move model (moves ALL parameters)
model = model.to(device)

# CRITICAL: Data and model must be on SAME device
# This CRASHES: model on GPU, data on CPU
output = model(x_cpu)  # RuntimeError!
```

P053 pattern: We use `get_device()` at the top of `train.py` and pass it everywhere. This lets the same code run on T4 (CUDA), A100 (CUDA), M1 Mac (MPS), or CPU. Hardware-agnostic code is a principal-level requirement.

2.4 Memory Layout (contiguous, stride, view vs clone)

```
# Contiguous = elements stored sequentially in memory
x = torch.randn(3, 4)           # Contiguous
x_t = x.t()                      # Transpose — NOT contiguous!
x_t.is_contiguous()              # False

# .view() requires contiguous memory
x_t.view(-1)                     # RuntimeError!
x_t.contiguous().view(-1)        # Works after making contiguous
x_t.reshape(-1)                  # Always works (may copy internally)

# Stride: how many elements to skip to advance one position
x = torch.randn(3, 4)
x.stride()    # (4, 1) — skip 4 elements for next row, 1 for next column
x.t().stride() # (1, 4) — transposed strides, NOT contiguous

# clone() vs detach()
y = x.clone()        # New memory, keeps grad connection
y = x.detach()       # Same memory, breaks grad connection
y = x.clone().detach() # New memory + no grad — safest for metrics
```

3. Autograd — Automatic Differentiation Engine

Autograd is PyTorch's engine for computing gradients. Every tensor operation is recorded in a **computation graph**. When you call `.backward()`, autograd traverses this graph in reverse (backpropagation) to compute gradients.

3.1 Computation Graph (Dynamic vs Static)

```
# PyTorch builds the graph AS YOU EXECUTE operations
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2 + 3 * x + 1      # Graph: x → x² → +3x → +1 → y
y.backward()                   # Compute dy/dx
print(x.grad)                  # tensor([7.0])  (2*2 + 3 = 7)

# DYNAMIC: The graph is rebuilt every forward pass
# This means you can use Python control flow!
def dynamic_model(x):
    if x.sum() > 0:             # Different path based on data
        return x * 2
    else:
        return x * 3
# TensorFlow 1.x couldn't do this without tf.cond!
```

> **Why "Dynamic"?** The graph is built on-the-fly during the forward pass, not defined beforehand. This makes debugging trivial (`print()` works everywhere), and enables architectures with variable-length inputs, conditional branches, and loops. This is why 80% of ML research uses PyTorch.

3.2 Gradient Accumulation, Retention, and Zeroing

```
# CRITICAL: Gradients ACCUMULATE by default
optimizer.zero_grad()           # Always zero before backward!
loss.backward()                  # Compute gradients
optimizer.step()                 # Update weights

# Gradient accumulation (simulate larger batch)
# If GPU can only fit batch=128, but we want effective batch=512:
accumulation_steps = 4
for i, (x, y) in enumerate(dataloader):
    loss = model(x, y) / accumulation_steps  # Scale loss
    loss.backward()                           # Accumulate grads
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()                      # Update
        optimizer.zero_grad()                 # Reset

# Gradient clipping (prevent exploding gradients)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Retain graph (for computing gradients twice)
loss.backward(retain_graph=True)  # Keep graph for second backward
```

Common bug: Forgetting `optimizer.zero_grad()`. Gradients accumulate across batches, causing the model to diverge. This is the #1 PyTorch beginner mistake. In P053, we zero gradients at the start of every training step.

3.3 Custom Autograd Functions

```
# For custom forward/backward (rare but powerful)
class FocalLossGrad(torch.autograd.Function):
    @staticmethod
    def forward(ctx, logits, targets, gamma=2.0):
        p = torch.sigmoid(logits)
        loss = -((1 - p) ** gamma) * targets * torch.log(p + 1e-8)
        ctx.save_for_backward(logits, targets, p)
        ctx.gamma = gamma
        return loss.mean()

    @staticmethod
    def backward(ctx, grad_output):
        logits, targets, p = ctx.saved_tensors
        # Custom gradient computation...
        return grad_logits, None, None
```

4. nn.Module — Building Neural Networks

Every model in PyTorch inherits from `torch.nn.Module`. It provides:

- Automatic parameter registration (for `optimizer`)
- `.to(device)` to move all parameters at once
- `.train()` / `.eval()` mode switching (affects Dropout, BatchNorm)
- `.state_dict()` for saving/loading
- Recursive module nesting

4.1 Layer Types

Layer	Shape	Used In P053?
`nn.Linear(in, out)`	(*, in) → (*, out)	✅ Feature embedding, MLP fusion
`nn.Conv1d(in_ch, out_ch, k)`	(B, C, L) → (B, C', L')	✅ Spatial features (wafer)
`nn.Conv2d(in_ch, out_ch, k)`	(B, C, H, W) → (B, C', H', W')	❌ (no image data)
`nn.TransformerEncoderLayer`	(S, B, D) → (S, B, D)	✅ Sensor feature attention
`nn.MultiheadAttention`	(L, B, D) → (L, B, D)	✅ Inside Transformer
`nn.Embedding(num, dim)`	int → (dim,)	❌ (continuous features only)
`nn.LayerNorm(dim)`	(*, D) → (*, D)	✅ Before attention
`nn.BatchNorm1d(features)`	(B, F) → (B, F)	✅ CNN branch
`nn.Dropout(p)`	Same shape	✅ 0.1 after fusion
`nn.GELU()`	Same shape	✅ Transformer activation
`nn.ReLU()`	Same shape	✅ CNN + MLP activation

4.2 Activation Functions

Function	Formula	When to Use
ReLU	$\max(0, x)$	Default for CNNs, simple MLPs. Can cause "dying neurons."
GELU	$x \cdot \Phi(x)$	Transformers (BERT, GPT). Smoother than ReLU. Used in P053 Transformer.
SiLU/Swish	$x \cdot \sigma(x)$	EfficientNet, modern architectures. Self-gated.
Mish	$x \cdot \tanh(\text{softplus}(x))$	YOLOv4. Slightly better than Swish.
Sigmoid	$1 / (1 + e^{-x})$	Binary classification output. Squashes to [0, 1].
Softmax	$e^{x_i} / \Sigma e^{x_j}$	Multi-class output. Squashes to probability distribution.
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	RNNs, LSTM. Output in [-1, 1].

4.3 Custom Modules and Forward Hooks

```
# Basic custom module pattern
class MyModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(0.1)

    def forward(self, x):
        x = F.relu(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        return self.fc2(x)

# Counting parameters
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad
print(f"Total: {total:,} | Trainable: {trainable:,}")
# P053 output: Total: 317,633 | Trainable: 317,633

# Forward hooks (inspect intermediate outputs)
def print_shape(module, input, output):
    print(f"{module.__class__.__name__}: {output.shape}")

model.fc1.register_forward_hook(print_shape)
```

4.4 P053 HybridTransformerCNN Architecture Deep Dive

> **Why Hybrid?** DRAM yield has TWO signal types: (1) sensor interactions (30+ electrical measurements with physics-driven correlations) and (2) spatial patterns (die position → edge effect failures). A single model can't capture both efficiently.

```python
# P053 Architecture (simplified from src/model.py)
class HybridTransformerCNN(nn.Module):
    """
    Transformer branch: 33 tabular features → per-feature embedding →
                        self-attention → CLS token → 128d
    CNN branch:         3 spatial features → Conv1d(1→32→64) →
                        GlobalAvgPool → FC → 64d
    Fusion:             concat(128 + 64) = 192d → MLP(192→128→64→1)
    """
    def __init__(self, n_tab=33, n_spatial=3, d_model=128,
                 n_heads=4, n_layers=2, dropout=0.1):
        super().__init__()

        # Per-feature tokenization (FT-Transformer approach)
        # Each of 33 features gets its own Linear(1→128) embedding
        self.feature_embed = nn.ModuleList([
            nn.Linear(1, d_model) for _ in range(n_tab)
        ])

        # CLS token (learnable, like BERT)
        self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=n_heads, dim_feedforward=256,
            dropout=dropout, activation='gelu', batch_first=True
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers
        )

        # CNN for spatial features
        self.spatial_cnn = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=1),
            nn.ReLU(),
            nn.Conv1d(32, 64, kernel_size=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool1d(1)  # Global average pooling
        )
        self.spatial_fc = nn.Linear(64, 64)

        # Fusion MLP
        self.classifier = nn.Sequential(
            nn.Linear(d_model + 64, 128),  # 128 + 64 = 192
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)               # Binary output (logit)
        )
```

```
    def forward(self, x_tab, x_spatial):
        # Tokenize each feature separately → (B, 33, 128)
        tokens = [emb(x_tab[:, i:i+1]) for i, emb in enumerate(self.feat
        tokens = torch.stack(tokens, dim=1)  # (B, 33, 128)

        # Add CLS token → (B, 34, 128)
        cls = self.cls_token.expand(x_tab.size(0), -1, -1)
        tokens = torch.cat([cls, tokens], dim=1)

        # Transformer self-attention
        encoded = self.transformer(tokens)  # (B, 34, 128)
        cls_output = encoded[:, 0, :]       # CLS token → (B, 128)

        # CNN spatial branch
        spatial = x_spatial.unsqueeze(1)    # (B, 1, 3)
        spatial = self.spatial_cnn(spatial).squeeze(-1)  # (B, 64)
        spatial = self.spatial_fc(spatial)   # (B, 64)

        # Fusion → prediction
        fused = torch.cat([cls_output, spatial], dim=1)  # (B, 192)
        logit = self.classifier(fused)      # (B, 1)

        return logit.squeeze(-1)            # (B,)
```

Interview answer: "P053 uses per-feature tokenization — each of 33 sensor features gets its own 128-dimensional embedding. This lets the Transformer's self-attention discover pairwise interactions like temperature×leakage (Arrhenius coupling) without manual feature engineering. The CNN branch captures spatial patterns (edge die failures cluster in concentric rings). The Fusion MLP combines both for a 317K-param model that beats XGBoost."

5. Training Loop — The Heart of Deep Learning

5.1 Loss Functions

Loss	Task	Formula / Notes
`nn.MSELoss`	Regression	Mean Squared Error: $\Sigma(\hat{y} - y)^2/n$
`nn.L1Loss`	Regression	Mean Absolute Error: $\Sigma\lvert\hat{y} - y\rvert/n$ (robust to outliers)
`nn.BCEWithLogitsLoss`	Binary class.	Sigmoid + BCE in one step (numerically stable)
`nn.CrossEntropyLoss`	Multi-class	Softmax + NLL. Takes raw logits + class indices
FocalLoss (custom)	Imbalanced	$-\alpha(1-p_t)^y \log(p_t)$. Down-weights easy negatives. **Used in P053**
`nn.TripletMarginLoss`	Metric learning	Anchor-positive-negative distance
`nn.CTCLoss`	Sequence	Connectionist Temporal Classification (speech)

```
# P053 FocalLoss implementation (from src/model.py)
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.75, gamma=2.0, label_smoothing=0.01):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.label_smoothing = label_smoothing

    def forward(self, logits, targets):
        # Label smoothing: hard 0/1 → 0.01/0.99
        targets_smooth = targets * (1 - self.label_smoothing) + self.lab
        bce = F.binary_cross_entropy_with_logits(logits, targets_smooth,
        pt = torch.exp(-bce)  # Probability of correct class
        focal_weight = (1 - pt) ** self.gamma  # Down-weight easy exampl
        alpha_weight = targets * self.alpha + (1 - targets) * (1 - self.
        loss = alpha_weight * focal_weight * bce
        return loss.mean()
```

5.2 Optimizers

Optimizer	Key Idea	When to Use
SGD	w -= lr × ∇L. With momentum: adds velocity	CNNs (ResNet). Simple, well-understood.
Adam	Adaptive LR per parameter (1st + 2nd moment)	Default choice. Good for most tasks.
AdamW	Adam + decoupled weight decay	Transformers, modern architectures. **Used in P053**
LAMB	Layer-wise Adaptive Moments (for large batch)	Distributed training, batch > 8K
LARS	Layer-wise Adaptive Rate Scaling	ResNet + large batch training
RAdam	Rectified Adam (auto warmup)	When LR warmup is critical

```
# P053 optimizer setup
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-3,
    weight_decay=1e-4,    # L2 regularization (decoupled)
    betas=(0.9, 0.999),   # Momentum terms
    eps=1e-8              # Numerical stability
)
```

5.3 Learning Rate Schedulers

Scheduler	Shape	Used in P053?
`StepLR(step=30, gamma=0.1)`	Drop ×0.1 every 30 epochs	❌ Too aggressive
`ExponentialLR(gamma=0.95)`	Multiply by 0.95 each epoch	❌ Too smooth
`CosineAnnealingLR`	Smooth cosine decay to 0	❌ No restarts
`CosineAnnealingWarmRestarts`	Cosine with periodic LR resets	✅ **Used in P053** (T_0=10, T_mult=2)
`OneCycleLR`	Warmup → peak → decay (one cycle)	❌ For single long training
`ReduceLROnPlateau`	Reduce when metric stops improving	❌ Reactive, not proactive

5.4 P053 Training Loop — Line by Line

```
# Simplified P053 training loop (from src/train.py)
model.train()
for epoch in range(num_epochs):
    running_loss = 0.0
    for batch_idx, (x_tab, x_spatial, labels) in enumerate(train_loader)
        # 1. Move data to GPU
        x_tab = x_tab.to(device)
        x_spatial = x_spatial.to(device)
        labels = labels.to(device)

        # 2. Zero gradients from previous step
        optimizer.zero_grad()

        # 3. Forward pass with AMP (mixed precision)
        with torch.amp.autocast('cuda', dtype=amp_dtype):
            logits = model(x_tab, x_spatial)
            loss = focal_loss(logits, labels)

        # 4. Backward pass
        if use_grad_scaler:   # float16 on T4
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(optimizer)
            scaler.update()
        else:                  # bfloat16 on A100 — NO scaler
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()

        running_loss += loss.item()

    # 5. Step LR scheduler
    scheduler.step()

    # 6. Validation (eval mode — disables Dropout, fixes BatchNorm)
    model.eval()
    with torch.no_grad():   # No gradient computation needed
        val_metrics = evaluate(model, val_loader, device, amp_dtype)
    model.train()            # Back to training mode

    # 7. Early stopping check
    if val_metrics['auc_pr'] > best_auc_pr:
        best_auc_pr = val_metrics['auc_pr']
        patience_counter = 0
        torch.save(model.state_dict(), 'best_model.pt')
    else:
        patience_counter += 1
        if patience_counter >= patience:
```

```
            print(f"Early stopping at epoch {epoch}")
            break
```

6. Data Pipeline — DataLoader, Dataset, Samplers

6.1 Custom Dataset Classes

```
# P053 Dataset (simplified)
class YieldDataset(torch.utils.data.Dataset):
    def __init__(self, features, spatial, labels):
        self.features = torch.FloatTensor(features)   # (N, 33)
        self.spatial = torch.FloatTensor(spatial)       # (N, 3)
        self.labels = torch.FloatTensor(labels)         # (N,)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.features[idx], self.spatial[idx], self.labels[idx]

# For HUGE datasets — IterableDataset (streaming)
class StreamingDataset(torch.utils.data.IterableDataset):
    """Reads Parquet files without loading everything into RAM"""
    def __init__(self, parquet_dir):
        self.files = sorted(Path(parquet_dir).glob('*.parquet'))

    def __iter__(self):
        for f in self.files:
            df = pd.read_parquet(f)
            for _, row in df.iterrows():
                yield torch.FloatTensor(row.values)
```

6.2 DataLoader Internals

```
train_loader = DataLoader(
    dataset,
    batch_size=512,           # Tuned for T4 16GB VRAM (see ED-008)
    shuffle=True,              # Randomize order each epoch
    num_workers=4,             # Parallel data loading processes
    pin_memory=True,           # Faster CPU→GPU transfer
    drop_last=True,            # Drop incomplete final batch
    persistent_workers=True    # Keep workers alive between epochs
)
```

Parameter	What It Does	P053 Value
`batch_size`	Samples per forward/backward pass	512 (T4: 6GB VRAM → fits 512 × 33 features)
`num_workers`	Parallel processes loading data	4 (g4dn.xlarge has 4 vCPU)
`pin_memory`	Pin CPU memory for faster DMA to GPU	True (always for CUDA training)
`persistent_workers`	Don't restart workers each epoch	True (saves 2-3s/epoch startup)
`prefetch_factor`	Batches to prefetch per worker	2 (default, sufficient for tabular)

6.3 Handling Class Imbalance

```
# WeightedRandomSampler (for small datasets < 1M)
pos_count = labels.sum()
neg_count = len(labels) - pos_count
weights = torch.where(labels == 1, 1.0/pos_count, 1.0/neg_count)
sampler = WeightedRandomSampler(weights, num_samples=len(labels))
loader = DataLoader(dataset, batch_size=256, sampler=sampler)

# P053 approach: FocalLoss instead (for 10M+ rows)
# WeightedRandomSampler with 10M weights causes OOM
# FocalLoss solves imbalance at the loss level — no sampling needed
```

7. GPU Training — CUDA, AMP, Multi-GPU

7.1 CUDA Fundamentals for ML Engineers

Concept	What It Means for PyTorch
Compute Capability	GPU architecture version. T4=7.5, A100=8.0, H100=9.0. Determines AMP support.
Tensor Cores	Specialized hardware for matrix multiply in FP16/BF16/TF32. 8× faster than regular cores.
VRAM	GPU memory. T4=16GB, A100=40/80GB. Determines max batch size.
CUDA Streams	Parallel GPU execution queues. PyTorch uses them automatically for async transfers.
Unified Memory	CPU+GPU share address space (A100+). Allows oversubscription.
nvcc	NVIDIA CUDA Compiler. Compiles custom CUDA kernels.

```
# Useful CUDA information
torch.cuda.get_device_name(0)           # 'Tesla T4'
torch.cuda.get_device_capability(0)     # (7, 5)
torch.cuda.get_device_properties(0).total_mem / 1e9  # 15.1 GB
torch.cuda.memory_allocated(0) / 1e9    # Current GPU memory used
torch.cuda.max_memory_allocated(0) / 1e9  # Peak GPU memory used
torch.cuda.empty_cache()                 # Free unused cached memory
```

7.2 Mixed Precision (AMP) — float16 vs bfloat16

dtype	Bits	Exponent	Mantissa	Best For
float32	32	8 bits	23 bits	Default, master weights
float16	16	5 bits	10 bits	T4, V100 (needs GradScaler)
bfloat16	16	**8 bits**	7 bits	A100, H100 (NO GradScaler needed)
tf32	19*	8 bits	10 bits	A100+ internal format (auto on matmul)

> **float16 danger:** With only 5-bit exponent, max representable value is ~65,504. FocalLoss gradients at 1:160 imbalance can exceed this → overflow → NaN → GradScaler death spiral. This is exactly what happened in P053 (see ED-031).

```
# P053 AMP pattern (hardware-adaptive)
cc = torch.cuda.get_device_capability()
if cc[0] >= 8:  # A100, H100
    amp_dtype = torch.bfloat16
    use_scaler = False  # bfloat16 doesn't need GradScaler
else:  # T4, V100
    amp_dtype = torch.float16
    use_scaler = True
    scaler = torch.amp.GradScaler('cuda')

# In training loop
with torch.amp.autocast('cuda', dtype=amp_dtype):
    logits = model(x_tab, x_spatial)
    loss = focal_loss(logits, labels)
```

7.3 GradScaler Deep Dive + P053 Death Spiral Bug

What GradScaler does:

1. Multiplies loss by a large scale factor (e.g., 65536) before `.backward()`
2. This amplifies small FP16 gradients so they don't underflow to zero
3. After backward, divides gradients by the scale factor to get true values
4. If NaN/Inf gradients detected → skips optimizer step, reduces scale by 0.5×

> **P053 Death Spiral (ED-031):**
> **v2 (float16):** GradScaler detected NaN at epoch 5 → halved scale → more NaN → halved again → scale reached 1.0 → underflow → all gradients zero → loss converged to constant 0.0693 (pure Focal Loss on all-negative prediction). **Model brain-dead.**
>
> **v3 (float16 + warmup):** LR warmup delayed the collapse to epoch 7. Same death spiral.
>
> **v4 (bfloat16):** 8-bit exponent handles the extreme gradient range. No GradScaler needed. Stable 50 epochs, AUC-ROC=0.816. **This is the champion.**

7.4 Multi-GPU (DataParallel, DDP, FSDP)

Method	How It Works	When to Use
`nn.DataParallel`	Replicates model to all GPUs, splits batch, gathers outputs	❌ Don't use — slow GIL bottleneck
`DDP`	One process per GPU. AllReduce gradients after backward	✅ Standard for multi-GPU training
`FSDP`	Shards model parameters across GPUs (ZeRO-3)	✅ When model doesn't fit in single GPU
DeepSpeed	ZeRO-1/2/3 + offloading + pipeline parallelism	✅ Billion-parameter models

```
# DDP setup (production standard)
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group(backend='nccl')
local_rank = int(os.environ['LOCAL_RANK'])
model = model.to(local_rank)
model = DDP(model, device_ids=[local_rank])
```

7.5 Apple MPS Backend

```
# Metal Performance Shaders (Apple Silicon GPU)
device = torch.device('mps') if torch.backends.mps.is_available() else '

# Limitations:
# - No AMP support (autocast not available for MPS)
# - Some operations fall back to CPU silently
# - ~10x slower than T4 for training (useful for prototyping only)
# P053: Used MPS for 3 epochs to benchmark, then moved to GPU
```

8. Model Saving, Loading, and Deployment

8.1 state_dict vs Entire Model Save

```
# ✅ RECOMMENDED: Save/load state_dict only
torch.save(model.state_dict(), 'model.pt')

# Load
model = HybridTransformerCNN(...)  # Must define architecture first
model.load_state_dict(torch.load('model.pt', map_location=device,
                                 weights_only=True))
model.eval()

# ❌ NOT recommended: Save entire model (includes code)
torch.save(model, 'model_full.pt')  # Brittle if code changes

# Save optimizer + scheduler (for resume training)
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict(),
    'best_auc_pr': best_auc_pr,
    'scaler_state_dict': scaler.state_dict() if use_scaler else None,
}
torch.save(checkpoint, f'checkpoint_epoch_{epoch}.pt')
```

> **Security:** Always use `weights_only=True` when loading models from untrusted sources. Without it, `torch.load()` uses pickle which can execute arbitrary code. This is a known attack vector.

8.2 TorchScript and torch.jit

```
# Convert PyTorch model to optimized format
# Trace-based (works for fixed control flow)
traced = torch.jit.trace(model, (example_tab, example_spatial))
traced.save('model_traced.pt')

# Script-based (works for dynamic control flow)
scripted = torch.jit.script(model)
scripted.save('model_scripted.pt')

# Load and run (no Python dependency!)
loaded = torch.jit.load('model_traced.pt')
output = loaded(x_tab, x_spatial)
```

8.3 ONNX Export

```
# Export to ONNX (universal format)
torch.onnx.export(
    model, (x_tab, x_spatial),      # Model + example inputs
    'model.onnx',
    input_names=['tabular', 'spatial'],
    output_names=['logit'],
    dynamic_axes={'tabular': {0: 'batch'}, 'spatial': {0: 'batch'}},
    opset_version=17
)
# Can now run on: ONNX Runtime, TensorRT, Qualcomm AI Engine, etc.
```

8.4 Quantization (INT8, INT4, QAT)

Method	Precision	Accuracy Loss	Speedup
Dynamic Quant.	FP32 → INT8 weights	~0.5%	2-3×
Static Quant.	INT8 weights + activations	~1-2%	3-4×
QAT	Quantization-aware training	~0.1%	3-4×
INT4 (GPTQ)	4-bit weights	~2-3%	4-6×

9. PyTorch 2.x — torch.compile and Beyond

9.1 TorchDynamo and TorchInductor

torch.compile (PyTorch 2.0+) is the biggest change since autograd. It captures Python code and compiles it into optimized GPU kernels.

- **TorchDynamo**: Captures Python bytecode → builds FX graph (the "tracer")
- **TorchInductor**: Compiles FX graph → optimized Triton/C++ kernels (the "compiler")
- Handles dynamic shapes, control flow, Python objects
- Typical speedup: 30-200% with zero code changes

9.2 torch.compile() — When and How to Use

```
# One-line speedup
model = torch.compile(model)  # That's it!

# With options
model = torch.compile(model, mode='reduce-overhead')  # Best for small m
model = torch.compile(model, mode='max-autotune')     # Best for large m
model = torch.compile(model, backend='inductor')       # Default backend

# Modes:
# 'default'          — Good balance (30-80% speedup)
# 'reduce-overhead'  — Minimize kernel launch overhead (small models/bat
# 'max-autotune'     — Try more kernel variants (slow compile, fast runt

# P053 note: Not used because model is tiny (317K params)
# and training is I/O bound, not compute bound. torch.compile
# shines at 10M+ parameter models.
```

9.3 Triton Integration (NVIDIA)

```
# TorchInductor generates Triton kernels automatically
# Triton = NVIDIA's Python-to-GPU compiler (easier than CUDA C++)

# You can also write custom Triton kernels
import triton
import triton.language as tl

@triton.jit
def fused_softmax_kernel(output_ptr, input_ptr, n_cols, BLOCK_SIZE: tl.c
    row_idx = tl.program_id(0)
    col_offsets = tl.arange(0, BLOCK_SIZE)
    mask = col_offsets < n_cols
    row = tl.load(input_ptr + row_idx * n_cols + col_offsets, mask=mask,
    row_max = tl.max(row, axis=0)
    numerator = tl.exp(row - row_max)
    denominator = tl.sum(numerator, axis=0)
    tl.store(output_ptr + row_idx * n_cols + col_offsets, numerator / de
```

10. Advanced Patterns for Production

10.1 Gradient Checkpointing (Memory vs Compute)

```
# Trade compute for memory — recompute activations during backward
from torch.utils.checkpoint import checkpoint

class MemoryEfficientModel(nn.Module):
    def forward(self, x):
        # Instead of storing all intermediate activations:
        x = checkpoint(self.layer1, x, use_reentrant=False)
        x = checkpoint(self.layer2, x, use_reentrant=False)
        return self.head(x)

# Saves ~50-70% memory at ~20-30% compute cost
# Essential for: LLMs, vision transformers, when batch size is tight
```

10.2 Custom CUDA Kernels

```
# Load custom CUDA kernel
from torch.utils.cpp_extension import load

custom_op = load(
    name='fused_focal_loss',
    sources=['csrc/focal_loss.cu'],
    verbose=True
)

# Or use torch.autograd.Function for custom backward
# P053 doesn't need this — FocalLoss in Python is fast enough for 317K p
```

10.3 Profiling with torch.profiler

```
from torch.profiler import profile, record_function, ProfilerActivity

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
    profile_memory=True,
    with_stack=True,
) as prof:
    with record_function("training_step"):
        output = model(x_tab, x_spatial)
        loss = focal_loss(output, labels)
        loss.backward()

# Print top operations
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=10)

# Export for Chrome trace viewer
prof.export_chrome_trace("trace.json")

# Export for TensorBoard
prof.export_stacks("stacks.txt", "self_cuda_time_total")
```

10.4 Debugging NaN/Inf Gradients

```
# Enable anomaly detection (slow but finds NaN source)
torch.autograd.set_detect_anomaly(True)

# Manual gradient checking
for name, param in model.named_parameters():
    if param.grad is not None:
        if torch.isnan(param.grad).any():
            print(f"NaN gradient in {name}")
        if torch.isinf(param.grad).any():
            print(f"Inf gradient in {name}")
        grad_norm = param.grad.norm()
        if grad_norm > 100:
            print(f"Exploding gradient in {name}: {grad_norm:.2f}")

# P053 debugging pattern: After the float16 collapse,
# we added gradient monitoring every 100 steps.
# This revealed GradScaler scale dropping to 1.0 → underflow.
```

11. PyTorch at AMD, NVIDIA, Qualcomm, Micron

AMD (ROCm)

- PyTorch runs on AMD GPUs via **ROCm** (Radeon Open Compute)
- Instinct MI300X (192GB HBM3) competes with H100
- Use: `pip install torch --index-url https://download.pytorch.org/whl/rocm6.0`
- **Interview relevance:** "I've profiled PyTorch on both CUDA (T4/A100) and understand ROCm compatibility. Our code uses `torch.cuda.is_available()` which works on both backends."

NVIDIA (CUDA, TensorRT, Triton)

- PyTorch is deeply integrated with CUDA. Tensor Core optimization is automatic with AMP.
- **TensorRT:** NVIDIA's inference optimizer. Export PyTorch → ONNX → TensorRT for 2-5× inference speedup
- **Triton:** NVIDIA's Python-to-GPU compiler. torch.compile uses it as backend.
- **NeMo:** NVIDIA's framework for LLMs, built entirely on PyTorch

Qualcomm (AI Engine, SNPE)

- Export PyTorch → ONNX → Qualcomm AI Engine (for mobile/edge)
- Qualcomm Neural Processing SDK (SNPE) supports INT8 quantized models
- Hexagon DSP runs inference at milliwatt power levels
- **Interview relevance:** "We export our model via ONNX for cross-platform deployment. The same P053 model could run on Snapdragon for edge inference at the fab."

Micron / Samsung / SK Hynix (Semiconductor)

- Custom PyTorch models for wafer yield prediction, defect classification
- Production models run on on-prem GPU clusters (DGX stations)
- Real-time inference at wafer test stations
- **P053 is directly relevant:** "This is the exact problem Micron solves — predicting DRAM die failures before electrical test using sensor and spatial features."

12. Complete API Reference — Most Used Functions

Tensor Creation

Function	Description
`torch.tensor(data)`	Create tensor from data (copies)
`torch.from_numpy(ndarray)`	Create tensor sharing NumPy memory
`torch.zeros/ones/full/eye`	Factory functions with specific values
`torch.randn/rand/randint`	Random tensors (normal/uniform/integer)
`torch.*_like(tensor)`	Create tensor with same shape/dtype
`torch.arange/linspace`	Sequential values

Tensor Operations

Function	Description
`torch.matmul(a, b)` / `a @ b`	Matrix multiplication
`torch.cat(tensors, dim)`	Concatenate along dimension
`torch.stack(tensors, dim)`	Stack along NEW dimension
`torch.where(cond, x, y)`	Element-wise conditional select
`torch.clamp(x, min, max)`	Clip values to range
`torch.sigmoid/softmax/relu`	Activation functions (functional)
`x.view/reshape/permute`	Shape manipulation
`x.sum/mean/max/min(dim)`	Reduction along dimension

Neural Network (torch.nn)

Module	Description
`nn.Linear(in, out)`	Fully connected layer: y = xW^T + b
`nn.Conv1d/2d(in, out, k)`	Convolution (1D for sequences, 2D for images)
`nn.TransformerEncoder`	Stack of self-attention + feedforward layers
`nn.Embedding(num, dim)`	Lookup table: integer → dense vector
`nn.LayerNorm/BatchNorm`	Normalization layers
`nn.Dropout(p)`	Randomly zero elements during training
`nn.Sequential(*layers)`	Chain layers sequentially
`nn.ModuleList(modules)`	List of modules (registered as parameters)

Training Utilities

Function	Description
`model.train() / model.eval()`	Toggle training/evaluation mode
`model.parameters()`	Generator of all trainable parameters
`model.state_dict()`	Ordered dict of all parameters (for saving)
`model.to(device)`	Move all parameters to device
`torch.no_grad()`	Context manager: disable gradient computation
`torch.amp.autocast()`	Mixed precision context manager
`torch.nn.utils.clip_grad_norm_`	Gradient clipping (prevent explosion)
`torch.save/load`	Serialize/deserialize (pickle-based)

13. Interview Questions with Answers

Q1: "What happens when you call loss.backward()?"

> **Answer:** PyTorch traverses the computation graph in reverse (topological sort), computing gradients via the chain rule. Each tensor with `requires_grad=True` accumulates gradients in its `.grad` attribute. The graph is destroyed after backward (unless `retain_graph=True`). This is called "reverse-mode automatic differentiation" — it computes all partial derivatives in a single backward pass, which is O(n) for n parameters.

Q2: "Why bfloat16 instead of float16 on A100?"

> **Answer:** bfloat16 has 8-bit exponent (same range as float32) vs float16's 5-bit exponent (max ~65K). In P053, FocalLoss with 1:160 imbalance generates extreme gradient values that overflow float16's range, causing GradScaler to repeatedly halve its scale factor until it reaches 1.0 — at which point gradients underflow to zero. bfloat16's wider range eliminates this entirely. Less precision in mantissa (7 vs 10 bits) doesn't matter for DNN training because SGD is inherently noisy.

Q3: "How would you scale this model to 1 billion parameters?"

> **Answer:** Four changes: (1) FSDP or DeepSpeed ZeRO-3 to shard parameters across GPUs — a 1B model needs ~4GB in fp32, fits on 2× A100 with FSDP. (2) Gradient checkpointing to trade compute for memory — saves ~60% activation memory. (3) torch.compile for kernel fusion and reduced overhead. (4) Streaming DataLoader (IterableDataset) because dataset won't fit in RAM. Infrastructure: switch from g4dn.xlarge to p4d.24xlarge (8× A100 80GB).

Q4: "What's the difference between model.eval() and torch.no_grad()?"

> **Answer:** `model.eval()` changes MODULE behavior: Dropout is disabled, BatchNorm uses running statistics instead of batch statistics. `torch.no_grad()` disables GRADIENT computation: no graph is built, saving memory and compute. For inference, you need BOTH: `model.eval()` for correct predictions, `torch.no_grad()` for efficiency. In P053, our validation loop uses both.

Q5: "How do you debug a model that produces NaN loss?"

Answer: Systematic approach: (1) `torch.autograd.set_detect_anomaly(True)` — this adds Python stack traces to the backward pass, showing exactly which operation produced NaN. (2) Check inputs: `torch.isnan(x).any()` before the model. (3) Check gradient norms after backward. (4) If using AMP, check GradScaler scale value — if it's dropping, your model is producing inf gradients. In P053, this is how we diagnosed the float16 collapse: GradScaler scale dropped from 65536 → 1.0 over 5 epochs.

Q6: "Explain your P053 architecture choice."

Answer: DRAM yield has two signal types requiring different inductive biases. Sensor features (33 electrical measurements) have complex pairwise interactions — temperature↔leakage follows Arrhenius physics, timing margins cascade across coupled paths. Self-attention in the Transformer branch discovers these interactions without manual feature engineering. Spatial features (3 wafer coordinates) have local structure — edge die failures cluster in concentric rings on the wafer. Conv1d captures this spatial locality. The Fusion MLP combines both 128d+64d representations. At 317K params, this outperforms XGBoost (0.054 vs 0.058 AUC-PR) while providing transfer learning capability for process node changes.

Q7: "How would you deploy this model on 100 AWS instances?"

Answer: Export to ONNX → TensorRT for inference optimization (2-5× speedup). Package in a Docker container with FastAPI endpoint. Deploy on ECS/EKS with HPA (horizontal pod autoscaler) scaling based on request latency. Model weights on S3, loaded at container startup. Blue-green deployment for zero-downtime updates. ALB (Application Load Balancer) for traffic routing. Prometheus + Grafana for latency/throughput monitoring. This is already designed in P053's Kubernetes manifests.